



Introduction to NumPy

You've learned that part of what makes Python such a powerful and dynamic language are the packages and libraries that are available to it. One of the most widely used and important of these is NumPy. Recall that NumPy contains multidimensional array and matrix data structures, as well as functions to manipulate them. You'll learn more about these data structures and functions soon, but for now, let's just consider NumPy at a high level and learn more about it. NumPy's power comes from vectorization.

Vectorization enables operations to be performed on multiple components of a data object at the same time. This is particularly useful for data professionals because their jobs often involve working with very large quantities of data, and vectorized code saves a lot of time because it computes more efficiently.

Let's explore this a little more. Suppose we have list A and list B, both the same length, and we want to create a new list C that's the element-wise product of each list. In other words, we want to multiply the first element of list A by the first element of list B, then multiply the second element of list A by the second element of list B, et cetera. If we try to multiply lists A and B together, the computer throws an error. To perform this operation, we could write a for loop. We'd start by defining an empty list, list C, then

make a range of indices to loop over, and append the product of list A and list B at each index to list C. This gets the job done, but if you're thinking, "There's gotta be an easier way," you're right!

We can use NumPy to perform this operation as a vectorized computation. Simply convert each list to a NumPy array and multiply the two arrays together using the product operator. The results are the same, but the vectorized approach is simpler, easier to read, and faster to execute because while loops iterate over one element at a time, vector operations compute simultaneously in a single statement. The efficiency of this might not be noticeable now, but when working with large datasets it will be. Vectors also take up less memory space, which is another factor that becomes important when working with a lot of data. You might have noticed that when we used NumPy, we first had to import it. This is called an import statement.

An import statement uses the import keyword to load an external library, package, module, or function into your computing environment. Once you import something into your notebook and run that cell, you don't need to import it again unless you restart your notebook. When we import NumPy, we import it as NP. This is known as aliasing.

Aliasing lets you assign an alternate name, or alias, by which you can refer to something. In this case, we're abbreviating NumPy to NP. Notice the NPs in the code below the import statement where we create our arrays.

If we didn't give NumPy an alias of NP, we'd have to type out "numpy" here in order to access its array function. Aliasing as NP makes the code shorter and easier to read. Note that NP is the standard alias. If you use something else, other people might get confused when reading your code. In addition to being highly useful in its own right,

NumPy powers a lot of other Python libraries, like pandas, so understanding how NumPy works will help you use these other packages. There's a lot more to discover about NumPy. Coming up, you'll learn about its core data structures and functionalities. See you soon.

Vectorization

Enables operations to be performed on multiple components of a data object at the same time

Import statement

Uses the `import` keyword to load an external library, package, module, or function into your computing environment

Aliasing

Lets you assign an alternate name—or alias—by which you can refer to something

Question



Fill in the blank: In NumPy, _____ enables operations to be performed on multiple components of a data object at the same time.

- ☐ classification
- ☐ evaluation
- ☒ vectorization
- ☐ conversion

✔ **Correct**

In NumPy, vectorization enables operations to be performed on multiple components of a data object at the same time. Data professionals often work with large datasets, and vectorized code helps them efficiently compute large quantities of data.

Continue

Skip